Bring Your Own Project (BYOP)

*Smart Route Finder using A Algorithm**

Fundamentals of AI and ML

CSA 2001 (C11+C12+C13)

BY KHUSHI PURWAR

25BCE10021

SATYENDRA SINGH

(Faculty)

Introduction

Pathfinding is a fundamental concept of computer science and artificial intelligence, with many diverse applications including navigation system, robotics and game development. It is the process of finding the shortest path for an agent such as a robot or video game character from a starting node to a target node without colliding with any obstacles. Path planning plays a key role in enhancing the performance and decision-making for real applications.

This project Smart Route Finder using A* Algorithm* is all about how to design and implement the A* (A-Star) algorithm along with the visualization in a 2D grid-based environment. The application enables users to choose a start point, an end point, and obstacles interactively, and then watch how the algorithm computes the shortest path.

A* is a heuristic search, which employs both the actual distance from the starting point and an estimated distance to the target to effectively calculate the best path. With logic and visualization, this project illustrates how intelligent systems take action in complex environments.

The goal this project is to be a very simple, interactive and visual pathfinding algorithm demonstration that can be used to better understand the basic A* algorithm and allows the user to experiment with different tree metrices.

Problem Statement

There are many practical applications for solution such as in navigation systems, robotics, and game developments where one need to find a shortest and efficient path from a starting point to an ending point avoiding some obstacles. However, it is not rare that people get confused on how such path finding system is implemented, as the logic is not visual at all.

Current solutions such as maps applications give answers, but don't reveal the underlying algorithmic process — a step that would help people understand, and grapple with, the decision-making tools used in ai.

Hence a simple, interactive and visual system that shows how the pathfinding algorithms work can be quite useful. This particular challenge is solved in this project which implement the A* algorithm on a two-dimensional grid where users can put start and end points as well as obstacles and watch how the algorithm find the route.

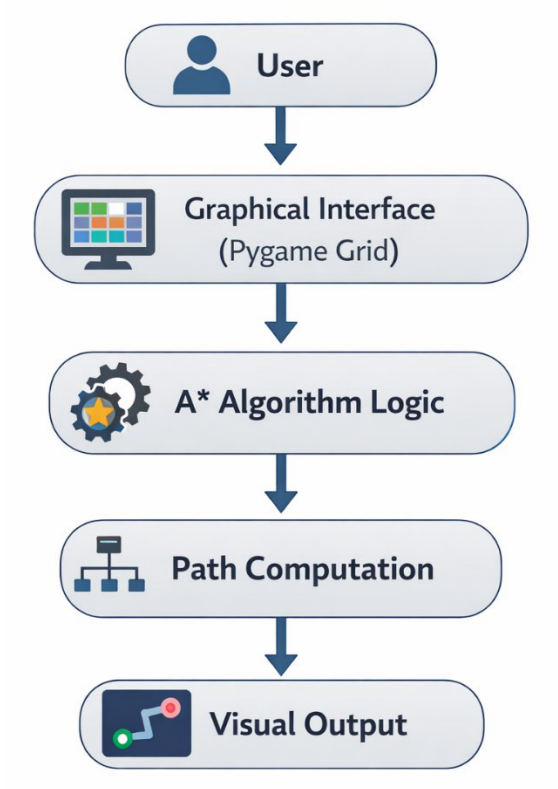
Functional Required

Requirement	Description
FR1	Select start node using mouse click
FR2	Select end node using mouse click
FR3	Place obstacles (walls) on the grid
FR4	Remove nodes using right-click
FR5	Execute A* algorithm using keyboard input (SPACE key)
FR6	Visualize algorithm step-by-step (open and closed nodes)
FR7	Display shortest path between start and end nodes
FR8	Reset the grid using keyboard input (C key)

Non – Functional Requirement

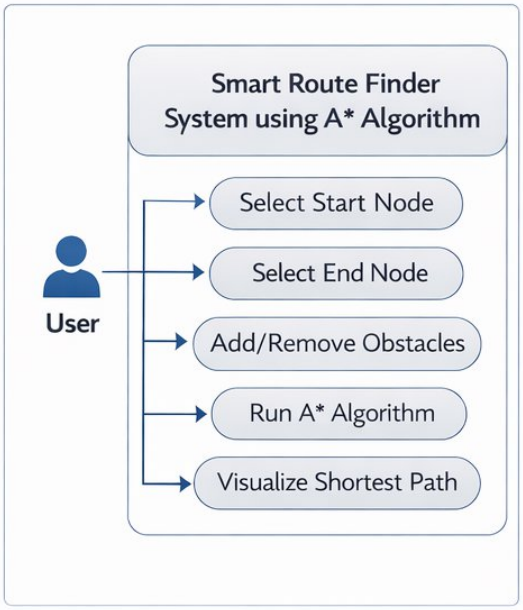
Requirement Type	**Expectation**
Usability	Simple and interactive graphical interface using mouse and keyboard
Performance	Fast execution of the A* algorithm with minimal delay
Reliability	Accurate and consistent pathfinding results
Portability	Runs on Windows, Linux, and macOS systems
Maintainability	Well-structured and readable code for easy updates
Scalability	Ability to increase grid size or add more features in future

System Architecture

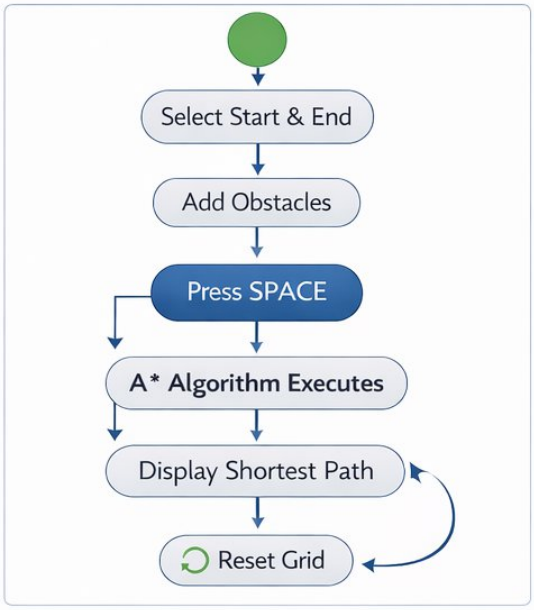


Design Diagrams

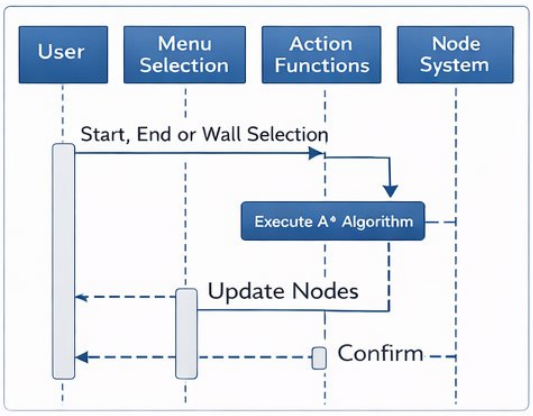
Use Case Diagram



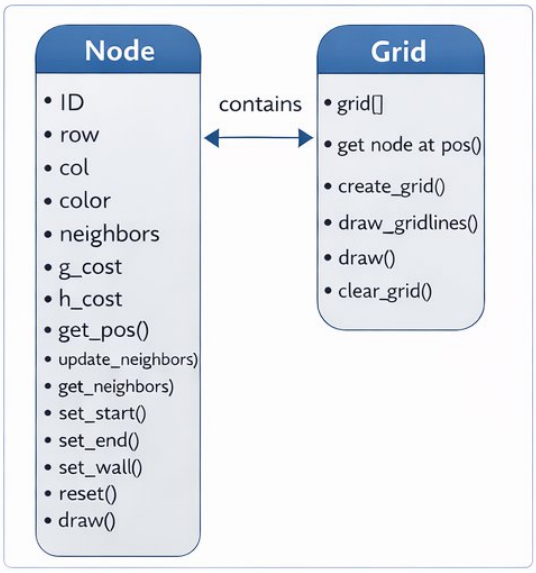
Workflow Diagram



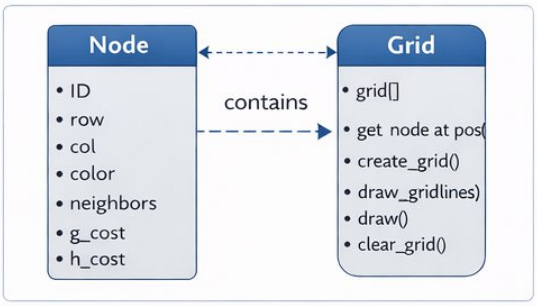
Sequence Diagram



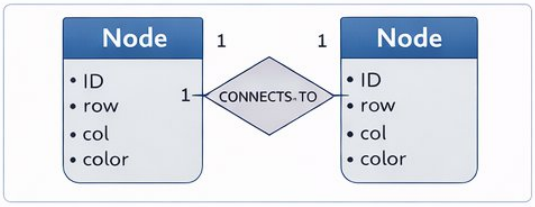
ER Diagram



Class/Component Diagram



ER Diagram



Design Decisions & Rationale

Design Decision	Rationale
Use of A* Algorithm	Chosen because it is efficient and guarantees the shortest path using heuristics
Grid-based Environment	Simplifies visualization and makes pathfinding easy to understand
Use of Manhattan Distance	Suitable for grid movement without diagonal directions
Pygame for GUI	Provides an interactive and visual representation of the algorithm
Color Coding System	Helps users easily distinguish between different node states
Priority Queue Implementation	Ensures nodes with lowest cost are explored first
Single File Implementation	Keeps the project simple and easier to understand for beginners
Event-driven Input Handling	Allows real-time interaction using mouse and keyboard

INPUT

astarfinal.py 1

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > ...

```python
import pygame
from queue import PriorityQueue

# SETTINGS
WIDTH = 600
ROWS = 30

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
GREY = (200, 200, 200)
GREEN = (0, 255, 0)
RED = (255, 0, 0)
BLUE = (0, 0, 255)
ORANGE = (255, 165, 0)
PURPLE = (128, 0, 128)

pygame.init()
WIN = pygame.display.set_mode((WIDTH, WIDTH))
pygame.display.set_caption("A* Pathfinding")

# NODE CLASS
class Node:
    def __init__(self, row, col, width):
        self.row = row
        self.col = col
        self.x = col * width   # ✅ FIXED
        self.y = row * width   # ✅ FIXED
        self.color = WHITE
        self.width = width

    def draw(self, win):
        pygame.draw.rect(win, self.color, (self.x, self.y, self.width, self.width))


# GRID
def make_grid():
    grid = []
```

⊗ 0 ⚠ 1

astarfinal.py 1

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > get_neighbors

```python

# GRID
def make_grid():
    grid = []
    gap = WIDTH // ROWS
    for i in range(ROWS):
        grid.append([])
        for j in range(ROWS):
            grid[i].append(Node(i, j, gap))
    return grid


def draw(win, grid):
    win.fill(WHITE)

    for row in grid:
        for node in row:
            node.draw(win)

    gap = WIDTH // ROWS
    for i in range(ROWS):
        pygame.draw.line(win, GREY, (0, i * gap), (WIDTH, i * gap))
        pygame.draw.line(win, GREY, (i * gap, 0), (i * gap, WIDTH))

    pygame.display.update()


# HEURISTIC
def h(p1, p2):
    x1, y1 = p1
    x2, y2 = p2
    return abs(x1 - x2) + abs(y1 - y2)

```

```python
# NEIGHBORS
def get_neighbors(node, grid):
    neighbors = []
    r, c = node.row, node.col

    if r < ROWS - 1 and grid[r + 1][c].color != BLACK:
        neighbors.append(grid[r + 1][c])
    if r > 0 and grid[r - 1][c].color != BLACK:
        neighbors.append(grid[r - 1][c])
    if c < ROWS - 1 and grid[r][c + 1].color != BLACK:
        neighbors.append(grid[r][c + 1])
    if c > 0 and grid[r][c - 1].color != BLACK:
        neighbors.append(grid[r][c - 1])

    return neighbors
```

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > h

```python

# A* ALGORITHM
def astar(draw_func, grid, start, end):
    count = 0
    open_set = PriorityQueue()
    open_set.put((0, count, start))

    came_from = {}
    g_score = {node: float("inf") for row in grid for node in row}
    g_score[start] = 0

    f_score = {node: float("inf") for row in grid for node in row}
    f_score[start] = h((start.row, start.col), (end.row, end.col))

    while not open_set.empty():
        current = open_set.get()[2]

        if current == end:
            while current in came_from:
                current = came_from[current]
                current.color = BLUE
                draw_func()
            return True

        for neighbor in get_neighbors(current, grid):
            temp_g = g_score[current] + 1

            if temp_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = temp_g
                f_score[neighbor] = temp_g + h(
                    (neighbor.row, neighbor.col),
                    (end.row, end.col)
                )
```

C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > h

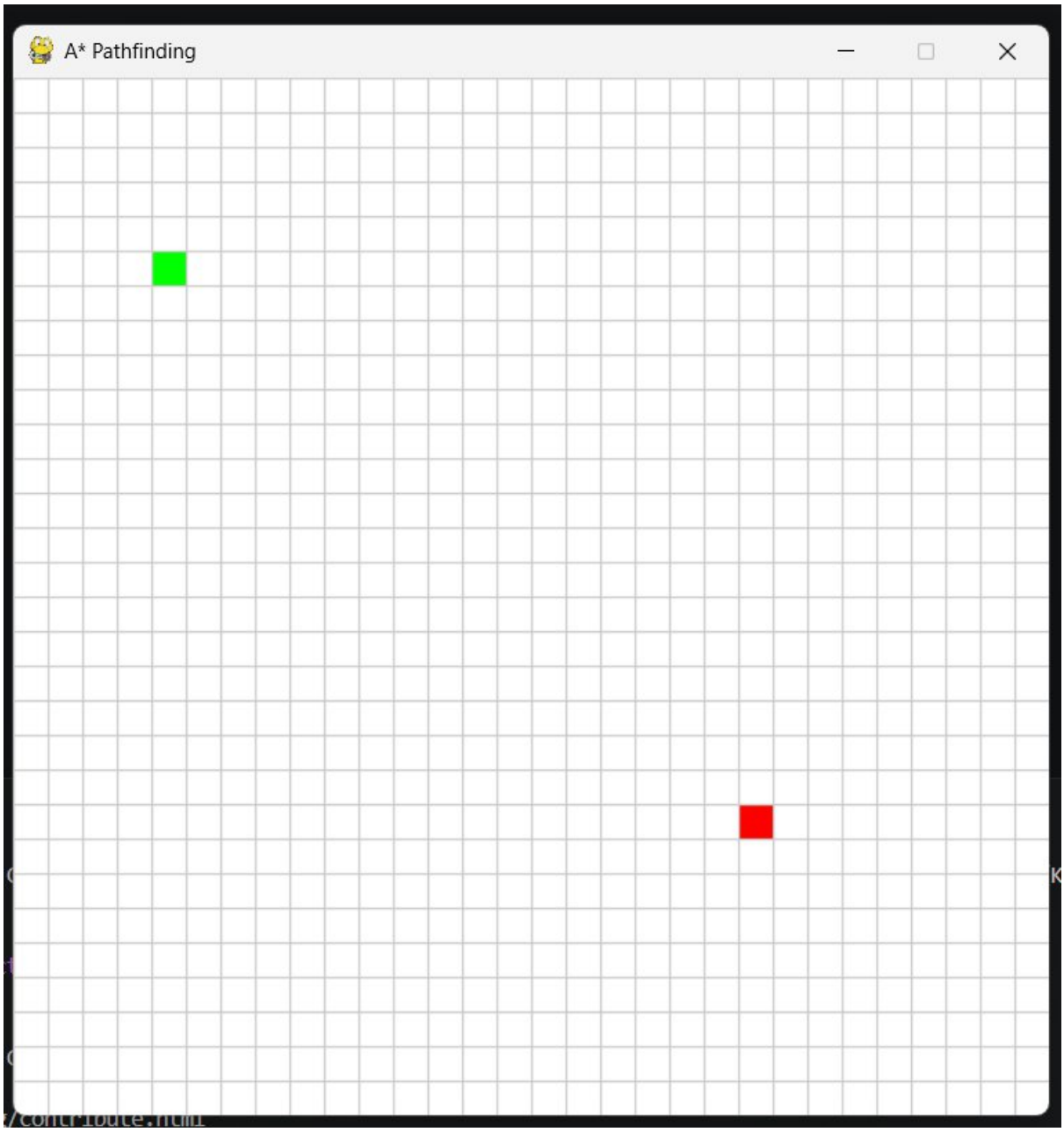
```python
def astar(draw_func, grid, start, end):
                if temp_g < g_score[neighbor]:
                    came_from[neighbor] = current
                    g_score[neighbor] = temp_g
                    f_score[neighbor] = temp_g + h(
                        (neighbor.row, neighbor.col),
                        (end.row, end.col)
                    )

                    count += 1
                    open_set.put((f_score[neighbor], count, neighbor))
                    neighbor.color = ORANGE

            if current != start:
                current.color = PURPLE

            draw_func()

        return False


# MAIN
def main():
    grid = make_grid()
    start = None
    end = None

    run = True
    while run:
        draw(WIN, grid)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False

```

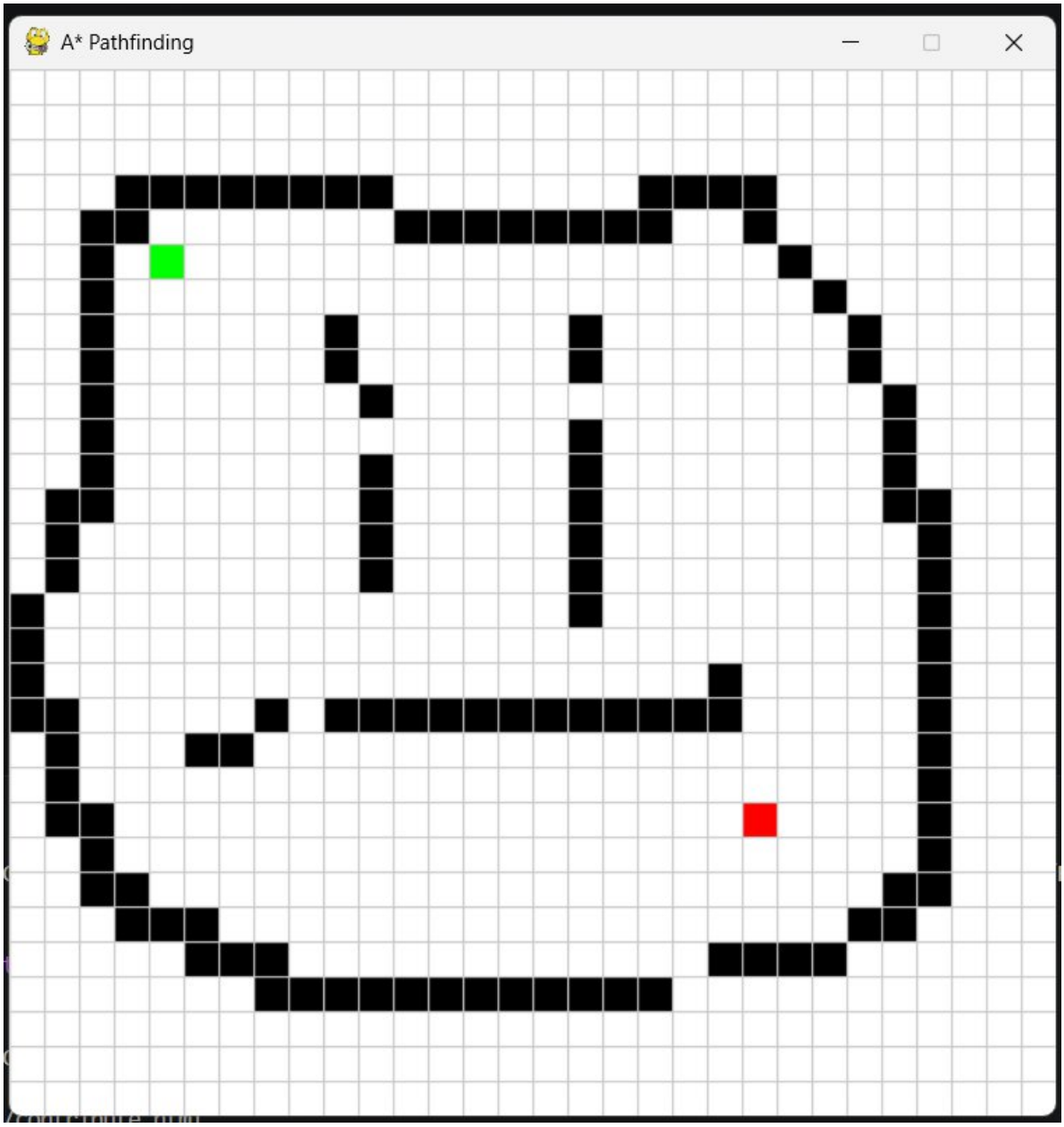
```python
# MAIN
def main():
    grid = make_grid()
    start = None
    end = None

    run = True
    while run:
        draw(WIN, grid)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE and start and end:
                    astar(lambda: draw(WIN, grid), grid, start, end)

                if event.key == pygame.K_c:
                    start = None
                    end = None
                    grid = make_grid()

            # LEFT CLICK
            if pygame.mouse.get_pressed()[0]:
                pos = pygame.mouse.get_pos()
                gap = WIDTH // ROWS
                x, y = pos
                row = y // gap
                col = x // gap

                node = grid[row][col]
```

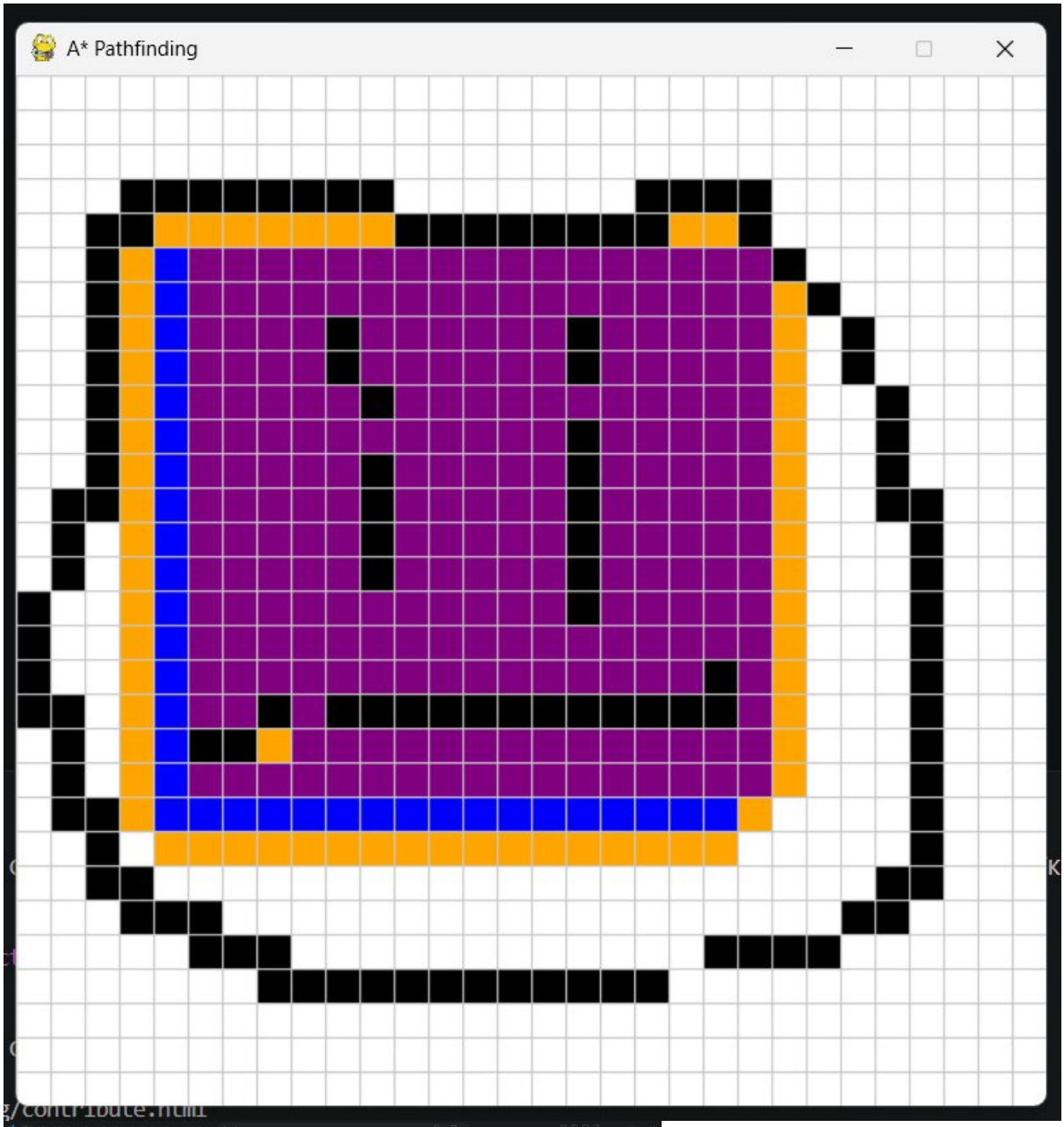
C: > Users > Khushi > OneDrive > Documents > astar_project > astarfinal.py > h

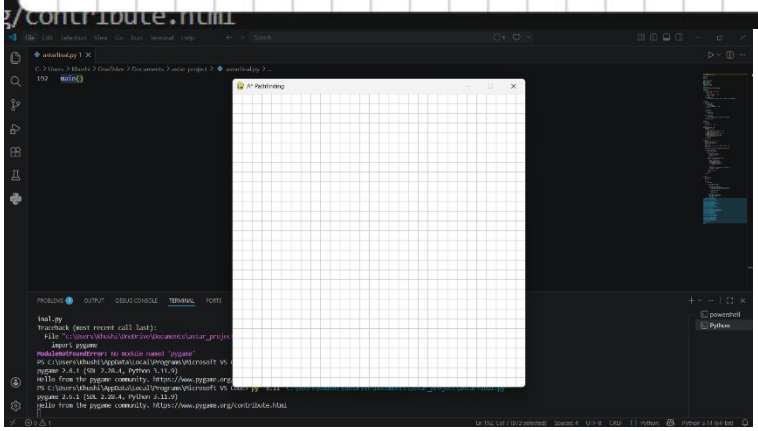
```python
def main():
                x, y = pos
                row = y // gap
                col = x // gap

                node = grid[row][col]

                if not start:
                    start = node
                    node.color = GREEN
                elif not end:
                    end = node
                    node.color = RED
                else:
                    node.color = BLACK

            # RIGHT CLICK
            elif pygame.mouse.get_pressed()[2]:
                pos = pygame.mouse.get_pos()
                gap = WIDTH // ROWS
                x, y = pos
                row = y // gap
                col = x // gap

                node = grid[row][col]
                node.color = WHITE

                if node == start:
                    start = None
                elif node == end:
                    end = None

    pygame.quit()


main()
```

OUTPUT




A* Pathfinding


A* Pathfinding


astarfinal.py
main()
A* Pathfinding
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
final.py
Traceback (most recent call last):
File "c:\users\khushi\onedrive\documents\asstar_projec
import pygame
ModuleNotFoundError: No module named 'pygame'
PS C:\Users\khushi\AppData\Local\Programs\Microsoft VS
pygame 2.6.1 (SDL 2.26.4, Python 3.11.9)
Hello from the pygame community. https://www.pygame.org
PS C:\Users\khushi\AppData\Local\Programs\Microsoft VS
pygame 2.6.1 (SDL 2.26.4, Python 3.11.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
powershell
Python

TESTING APPROACH

Test Type	Description
Functional Testing	Verified that all features like start/end selection, obstacle placement, and pathfinding work correctly
User Input Testing	Checked mouse clicks (left/right) and keyboard inputs (SPACE, C) for proper response
Boundary Testing	Tested grid edges to ensure no out-of-bound errors occur
Performance Testing	Ensured the algorithm runs smoothly without lag for different grid setups
Algorithm Testing	Validated that A* always finds the shortest path when possible
Visualization Testing	Confirmed correct color changes for nodes during execution
Reset Testing	Verified that pressing 'C' clears the grid properly

Challenges Faced

At various stages of this project, in a arcane parallel universes of challenge and triumph, we met with many obstacles. A typical such problem was that of implementing the priority queue in Python, where errors of comparison would be raised if two nodes had the same priority value. This was sorted out by adding a tie-breaker breaker variable to the ordering.

Another challenge was to accurately convert mouse clicks to grid coordinates. At first there were errors as a result of a mishandling of x and y values when placing nodes that made them misalign. Fix: Convert screen coordinates to the grid indices appropriately.

Deciphering the solutions to run time visualization with the algorithm was another hurdle. The event loop had to be tightly controlled, with the screen refreshing constantly as the algorithm was progressing.

Furthermore, time and effort was spent in fixing the problems with pathing for files, and with python versions and libraries installations (pygame). These were fixed by setting the right python environment and getting/setting the right file execution path.

At the end of the day, all these challenges were useful in training up my debugging skills and my knowledge for both programming and algorithm implementation.

Learnings & Key Takeaways

- Got a clear idea of A* (A-Star) algorithm and how heuristic based search works
- How shortest path problems are solved in practice in products like GPS and google maps
- Gained Hands-on experience in Python.
- Understand that certain types of data (e.g., priority queues) can be used to achieve efficient computation.
- I also learned to create interactive graphical applications (games) with the Pygame module, that's cool!
- Improved ability to debug, finding and fixing logic-, coordinate-, and environment-related errors.
- Realize the need for visualization, especially when complex algorithms need to be explained.
- Learned how to properly lay out a project and document it.

Future Enhancements

- Enable diagonal movement so that pathfinding is more realistic.
- Add heurstics (Euclidean distance) etc. for comparison.
- Introduce speed control to change visualisation speed of the algorithm.
- Enhance the user interface for improved design and ease of use.
- Support dynamic grid resizing to cover larger and more complex areas.
- Add animation (for smoother visualization).
- Port the project to real-world map data.
- Add AI-based best-route recommendations.
- Create a web/mobile version to make it more accessible.
- Include obstacle generation capabilities to simulate various scenarios.

Conclusion

With this project we want to give the proof of concept A* (A-Star) algorithm can be used to solve shortest track/path problem in a two-dimensional grid. By mixing algorithmic logic with animated visualization, the system makes it easy to understand how intelligent pathfinding is used in real applications.

The project emphasis the relevance of heuristic based search method for increasing efficiency and accuracy. The implementation illustrates how A* makes an intelligent search of nodes and calculate the best path circumventing obstacles.

In general, this project has been very useful to refresh and reinforce some important concepts related to artificial intelligence, problem solving and python programming. It also demonstrated the importance of visualization to make complex algorithms more friendly to the user.